Redis, queue và Kafka

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Redis: cache hay nguồn dữ liệu?

Redis thường được dùng làm cache, session store, queue nhẹ hoặc cấu trúc dữ liệu trong bộ nhớ. Khi là cache, dữ liệu gốc nằm nơi khác; mất Redis có thể làm hệ thống chậm nhưng không mất dữ liệu nghiệp vụ nếu thiết kế đúng. Khi dùng Redis như nguồn dữ liệu chính, persistence, replication và backup trở thành yêu cầu khác hẳn. Redis có các chế độ persistence như snapshot RDB và append-only file; lựa chọn liên quan độ bền và chi phí ghi.

Cache tạo hai câu hỏi khó: khi dữ liệu gốc thay đổi, cache được cập nhật/xóa thế nào; nếu cache lỗi hoặc trả dữ liệu cũ, người dùng thấy gì? TTL giúp giới hạn thời gian stale nhưng không giải quyết mọi trường hợp. Đừng thêm Redis trước khi đo vấn đề cần nó giải quyết.

Queue và event stream

RabbitMQ/ActiveMQ thường dùng broker/queue để tách producer khỏi consumer. Producer gửi message; consumer xử lý; cần hiểu ack, retry, dead-letter queue và idempotency. Nếu consumer xử lý xong nhưng chết trước khi ack, message có thể được giao lại; logic xử lý phải chịu được việc lặp. Kafka là distributed event log: topic chia partitions, consumer group đọc theo offset, và nhiều nhóm có thể đọc cùng dữ liệu theo tiến độ riêng. Thứ tự được bảo đảm trong phạm vi partition theo cấu hình, không tự động trên toàn topic.

Kafka Connect nối nguồn/đích dữ liệu với Kafka qua connector. Nó không biến mọi tích hợp thành "không cần code": connector có cấu hình, schema, offset, quyền và lỗi. RabbitMQ và Kafka không hoán đổi đơn giản chỉ vì đều truyền message. Hỏi nhu cầu: job queue, pub/sub, replay sự kiện, throughput, ordering, retention, vận hành và chi phí.

Bài tập thiết kế xử lý ảnh

API nhận yêu cầu tạo thumbnail. Nếu làm đồng bộ, request chờ worker và dễ timeout. Với queue, API ghi trạng thái **pending**, gửi message, worker tạo ảnh rồi cập nhật **done**. Nếu worker lỗi, message retry; nếu lỗi vĩnh viễn, đưa vào dead-letter queue hoặc trạng thái lỗi để người vận hành thấy. Hãy xác định nơi lưu file gốc, message ID, cách tránh tạo thumbnail trùng và metric về số message chờ. Đây là bài tập áp dụng chung trước khi chọn RabbitMQ, Kafka hay Service Bus.

Tính nhất quán khi DB và message cùng tham gia

Request tạo đơn hàng có thể cần ghi DB rồi gửi message. Nếu ghi DB thành công nhưng gửi thất bại, worker không biết đơn hàng mới. Nếu gửi trước rồi DB rollback, worker xử lý order không tồn tại. Transactional outbox ghi bản ghi nghiệp vụ và một hàng outbox trong **cùng transaction DB**; publisher riêng gửi message. Publisher có thể gửi lặp nếu chết sau khi gửi nhưng trước khi đánh dấu, nên consumer cần idempotency và metric số outbox tồn đọng.

Vẽ bảng **orders(id,status)** và **outbox(id,event_type,payload,published_at)**. Mô tả transaction tạo order, publisher retry, consumer nhận cùng event hai lần và cơ chế tránh xử lý trùng. Theo dõi số hàng

chưa publish, tuổi hàng lâu nhất, dead-letter và latency tới khi hoàn tất. Đừng hứa "exactly once" trên toàn chuỗi nếu chưa chỉ ra ranh giới và cơ chế chống lặp.

Chọn queue hay Kafka theo yêu cầu

Nếu cần phân phối công việc cho worker với ack/retry/dead-letter, queue như RabbitMQ/Service Bus thường tự nhiên. Nếu nhiều nhóm độc lập cần đọc cùng dòng sự kiện và phát lại lịch sử theo offset, Kafka có mô hình phù hợp. Các hệ có vùng chồng lắp; quyết định dựa vào ordering, retention, throughput, latency, kỹ năng đội và chi phí vận hành.

Với Kafka, partitions ảnh hưởng song song và thứ tự. Nếu cần thứ tự của **một order**, dùng key nhất quán để cùng order vào cùng partition; không hứa thứ tự giữa mọi order. Consumer lag tăng có thể do input tăng, consumer lỗi hoặc downstream chậm. Khi alert lag, xem cả tốc độ input, tốc độ xử lý và tuổi event; một con số lag đơn lẻ không luôn nói tác động người dùng.

Case study có lời giải: đơn hàng trùng sau retry

Bối cảnh: worker tạo vận đơn từ message; hệ thống thấy hai vận đơn cho cùng order sau một lần consumer restart. Có thể worker đã gọi dịch vụ vận đơn thành công rồi chết trước khi ack, broker giao lại. Chỉ tăng timeout hoặc tắt retry sẽ che vấn đề và có thể làm mất việc. Kiểm tra message ID/order ID, thời điểm xử lý, ack log và trạng thái downstream. Thiết kế idempotency key cố định theo order + thao tác, lưu kết quả xử lý hoặc yêu cầu downstream hỗ trợ key; khi nhận lại, trả cùng kết quả thay vì tạo vận đơn mới. Cần kiểm chứng tình huống chết **ngay sau side effect nhưng trước ack**, vì đó là cửa sổ lỗi quan trọng.

Theo dõi: số retry, dead-letter, tuổi message, tỷ lệ duplicate bị chặn và đơn hàng chưa hoàn tất. Nếu muốn xóa vận đơn trùng, đó là quy trình nghiệp vụ/đối soát, không chỉ sửa queue. Bài này cho thấy tại sao hiểu transaction và application behavior quan trọng ngang việc cài broker.

Từ request đồng bộ sang công việc bất đồng bộ

API nhận yêu cầu tạo thumbnail, ghi trạng thái **pending**, rồi đưa message vào broker. Worker lấy message, xử lý file, cập nhật **done** hoặc **failed**. Điều này giảm thời gian chờ của request nhưng thêm trạng thái, retry, dead-letter và quan sát backlog. Producer cần quyết định điều gì xảy ra khi broker không nhận message; consumer cần quyết định khi xử lý xong nhưng ack thất bại. Message ID và idempotency key phải đi qua các lớp để đối soát.

Redis cache khác queue: cache giữ bản sao dữ liệu để đọc nhanh, có TTL/invalidation; queue giữ công việc cần xử lý. Redis có cấu trúc để làm queue nhưng yêu cầu durability/retry/ordering cần đánh giá theo cách dùng. Khi cache lỗi, hệ thống có thể fallback DB nhưng phải tránh stampede làm DB quá tải. Nếu Redis giữ session hoặc dữ liệu gốc, mất dữ liệu có tác động lớn hơn cache đơn thuần; persistence/replication/backup cần thiết kế tương ứng.

RabbitMQ và Kafka: câu hỏi lựa chọn

Với RabbitMQ, xem exchange/queue/binding, ack, prefetch, retry và dead-letter; mỗi job có thể được phân phối tới một worker trong nhóm tùy topology. Với Kafka, event log chia partition, consumer group theo offset, nhiều nhóm độc lập có thể replay. Ordering trong một partition theo key; không hứa global order. Kafka Connect giúp chuyển dữ liệu nhưng connector/offset/schema/quyền vẫn cần vận hành. Chọn theo yêu cầu replay/retention/throughput/ordering, độ phức tạp và năng lực đội. Đừng thêm Kafka cho vài job nhỏ chỉ vì JD có chữ Kafka.

Diễn tập duplicate delivery

Worker gọi API vận đơn thành công rồi chết trước ack. Broker giao lại message. Nếu worker tạo vận đơn mới, khách có hai vận đơn. Thiết kế key theo **order_id + action**, lưu kết quả đã xử lý và cho downstream nhận key nếu hỗ trợ. Trong test, tạo lỗi đúng ở cửa sổ sau side effect trước ack. Đo duplicate bị chặn, retry, dead-letter và tuổi message; sau sự cố đối soát kết quả nghiệp vụ, không chỉ xóa message khỏi queue.

Backpressure và alert

Nếu producer gửi 1000 message/phút còn worker xử lý 700, backlog tăng 300/phút. Tăng worker chỉ hữu ích nếu downstream chịu được; nếu DB đã bão hòa, tăng concurrency làm tệ hơn. Alert nên xem tuổi message lâu nhất, backlog theo thời gian, throughput input/output và lỗi xử lý. Một queue depth 1000 có thể bình thường khi worker nhanh; 100 message cũ một giờ có thể nghiêm trọng. Với Kafka, consumer lag cần gắn với tốc độ vào/ra và event age để hiểu tác động.

Vì sao cần queue thay vì gọi đồng bộ?

Một request tạo đơn hàng có thể cần gửi email, cập nhật search index và gọi đối tác. Nếu làm tuần tự trong HTTP request, latency và khả năng lỗi phụ thuộc mọi downstream. Queue tách thời điểm producer tạo công việc khỏi consumer xử lý. Producer ghi message và trả response theo contract; consumer nhận, xử lý, xác nhận hoặc retry. Điều này không làm mất lỗi: nó chuyển lỗi sang backlog/dead-letter và đòi observability, idempotency, retention và ownership rõ ràng.

RabbitMQ phù hợp nhiều kiểu routing/queue công việc; Kafka là log phân vùng cho stream/event có retention và consumer group. Chọn theo nhu cầu ordering, replay, throughput, số consumer và vận hành; không nói Kafka “mạnh hơn” nên luôn dùng. Redis có thể là cache, cấu trúc dữ liệu và một số kiểu message/stream, nhưng phải quyết định persistence/eviction/replication và dữ liệu nào là nguồn sự thật. Một cache miss phải có đường tới source; cache stale có thể gây kết quả sai nếu invalidation kém.

Delivery semantics bằng timeline

Consumer nhận message **order-42**, ghi DB thành công, rồi chết **trước ack**. Broker có thể phát lại message. Nếu handler chèn đơn mới mỗi lần, tạo trùng. “At least once” nghĩa thiết kế idempotency, ví dụ message ID/unique constraint và transaction để chỉ áp hiệu ứng một lần. Nếu ack **trước** khi ghi DB rồi process chết, công việc có thể mất. “Exactly once” thường có phạm vi và giả định cụ thể; không dùng khẩu hiệu thay phân tích giao dịch qua DB và broker. RabbitMQ docs giải thích ack là chuyển trách nhiệm nhận/xử lý theo semantics của consumer; xem reliability guide.

Ordering cũng có phạm vi. Kafka giữ thứ tự trong một partition; nếu key order ID được đưa tới cùng partition, event của một order có thứ tự tương đối ở đó, nhưng không có thứ tự toàn cục giữa mọi partition. Thêm consumer/partition để scale có thể đổi cách phân phối; retry/dead-letter có thể làm event tới muộn. Handler nên kiểm tra version/timestamp và chấp nhận duplicate/out-of-order theo bài toán. Với RabbitMQ, nhiều consumer/prefetch/requeue cũng ảnh hưởng thứ tự quan sát.

DB và message: bài toán dual write

API ghi DB thành công rồi publish message thất bại: order tồn tại nhưng không ai gửi email. Publish trước rồi DB commit thất bại: email được gửi cho order không tồn tại. Transaction giữa hai hệ thống khác nhau không tự nhiên có atomicity. **Transactional outbox** ghi entity và outbox event trong cùng DB transaction; worker đọc outbox và publish, đánh dấu tiến độ. Nó giảm lỗ hổng dual write nhưng vẫn có thể publish lặp khi worker chết sau publish trước đánh dấu; consumer vẫn phải idempotent. Đây là mô hình quan trọng cho JD Kafka/RabbitMQ/Kafka Connect và microservices.

Lab trên giấy và lab local tùy chọn

Bài giấy: vẽ producer -> broker -> consumer -> DB cho 1000 message/phút. Consumer xử lý 600/phút: backlog tăng 400/phút, sau 30 phút là 12.000 message nếu tốc độ không đổi. Thêm hai consumer mỗi cái 600/phút có thể giảm backlog, nhưng DB downstream có chịu nổi 1200 ghi/phút không? Tính age of oldest message, không chỉ count. Một queue dài nhưng message vừa đến có ý nghĩa khác backlog chờ hàng giờ.

Lab local khi đã có Docker: chạy broker riêng theo tài liệu chính thức trong môi trường lab, tạo một producer/consumer nhỏ có message ID và DB giả/SQLite. Gửi message, cố tình cho consumer chết sau ghi nhưng trước ack, quan sát redelivery và dùng unique constraint để tránh bản ghi trùng. Ghi kết quả thực, phiên bản broker và chính sách ack; không giả định behavior giống nhau giữa RabbitMQ/Kafka. Dọn container/volume lab sau khi xác nhận đúng tên; nếu muốn giữ dữ liệu làm bằng chứng, export trước khi xóa.

Monitoring và runbook

Theo dõi publish/consume rate, backlog, age, retry, dead-letter, consumer lag, broker disk/memory/network và error downstream. Alert “queue length > 1000” thiếu ngữ cảnh: với 10.000 msg/s có thể bình thường, với 10 msg/s có thể nhiều giờ chậm. Đặt threshold theo SLO xử lý, capacity và tốc độ thay đổi. Khi backlog tăng, kiểm tra consumer còn sống, lỗi DB, poison message, partition skew, quota và recent deploy; không tăng số consumer vô hạn nếu DB đang nghẹt.

Câu hỏi: cache Redis có thể là nguồn dữ liệu duy nhất cho đơn hàng không? Chỉ khi kiến trúc/persistence/backup được thiết kế cho yêu cầu ấy; thông thường dữ liệu nghiệp vụ bền cần source khác. **Consumer ack sau ghi DB mà vẫn có duplicate?** Có, nếu chết giữa ghi và ack. **Kafka lag 0 nghĩa chức năng đúng?** Không; consumer có thể xử lý sai nhưng commit offset. Nguồn: Kafka documentation, RabbitMQ docs, Redis docs.

Poison message và retry storm

Một message có dữ liệu sai khiến consumer luôn exception. Nếu broker requeue ngay lập tức, cùng message chạy lặp hàng nghìn lần/giây, chiếm CPU/log và chặn message tốt. Thiết kế retry có số lần/độ trễ/backoff, phân biệt lỗi tạm thời (DB timeout) với lỗi vĩnh viễn (schema invalid), đưa poison message vào dead-letter sau giới hạn và alert có owner. DLQ không phải thùng rác quên đi: cần xem, sửa dữ liệu/code rồi replay có kiểm soát, tránh tạo duplicate side effect. Nếu consumer commit offset/ack sai thời điểm, message có thể mất hoặc xử lý lặp.

Backpressure và downstream

Nếu producer 2000 msg/s, consumer 1000 msg/s, backlog tăng 1000 msg/s. Sau 15 phút có 900.000 message nếu tốc độ giữ nguyên. Tăng consumer từ 2 lên 4 có thể giúp khi bottleneck là CPU consumer; nếu DB chỉ ghi được 1000/s, tăng consumer làm DB timeout và retry, có thể còn tệ hơn. Đo processing rate, lag age, error/retry, DB capacity và partition/queue distribution. Rate limit producer hoặc ưu tiên loại message quan trọng có thể là biện pháp giảm tác động.

Outbox có lời giải theo failure point

Worker đọc outbox row, publish Kafka thành công, chết trước khi đánh dấu row đã publish. Sau restart, nó publish lại: consumer phải dedupe theo event ID/transaction. Nếu đánh dấu row trước rồi publish thất bại, event có thể mất; vì thế thứ tự và retry phải được thiết kế. Outbox không loại bỏ mọi duplicate; nó giúp không bỏ sót event do dual write giữa DB và broker theo điều kiện triển khai. Giám sát outbox age và số row pending, không chỉ broker lag.

Chọn key/partition cho Kafka

Nếu cần thứ tự event của từng order, dùng order ID làm key để các event của order vào cùng partition theo cơ chế partitioner của hệ thống. Nhưng một order rất “nóng” có thể tạo hot partition; tăng partition không tự chia event của cùng key. Nếu dùng key ngẫu nhiên để cân tải, mất thứ tự per order. Đây là tradeoff thiết kế. Consumer group có thể scale tới số partition hữu ích; nhiều consumer hơn partition không tăng parallelism cho topic đó theo mô hình thông thường.

Câu hỏi: queue rỗng nhưng người dùng không nhận email, có thể lỗi gì? Producer không publish, consumer ack trước gửi, downstream email lỗi nhưng handler nuốt lỗi, hoặc đang xem sai queue/tenant; cần trace/correlation. **Kafka lag tăng, CPU consumer thấp?** Có thể consumer chờ DB/network, partition skew, poll loop bị block hoặc rebalance; CPU thấp không đủ kết luận. Nguồn: Kafka documentation, RabbitMQ reliability.

Bài tập chọn công cụ theo yêu cầu

Tác vụ A: gửi email xác nhận sau khi tạo note, retry có giới hạn, không cần replay lịch sử cho nhiều nhóm consumer. Queue kiểu RabbitMQ có thể phù hợp; cần ack, DLQ, idempotency và monitoring. **Tác vụ B:** nhiều đội cần đọc luồng event thay đổi đơn hàng trong 30 ngày, replay để xây lại projection. Kafka/log event có thể phù hợp nếu đội vận hành được partitions, retention, schema và consumer lag. **Tác vụ C:** cache dữ liệu note đọc nhiều, có thể tái tạo từ DB. Redis là ứng viên; cần TTL/invalidation và bảo vệ DB khỏi cache stampede. Những lựa chọn này chỉ là khởi điểm: traffic, compliance, ordering và kỹ năng đội có thể đổi quyết định.

Khi phỏng vấn, đừng trả lời bằng benchmark “Kafka nhanh hơn RabbitMQ”. Hỏi delivery semantics, replay, fan-out, latency, operations và failure recovery. Một hệ thống có thể dùng cả hai vì trách nhiệm khác nhau, nhưng mỗi broker mới là một dịch vụ stateful cần backup/upgrade/alert.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://www.rabbitmq.com/docs
- https://kafka.apache.org/documentation/
- https://redis.io/docs/latest/